

Task2: MovieLens + SQLite Analysis Assignment

This notebook performs the following tasks:

1. Build a SQLite database and import `movies.csv` and `ratings.csv`
2. Create indexes for analysis queries
3. Run 5 required SQL analysis tasks and display results
4. Export each query result to CSV
5. Validate outputs with assertions

```
In [1]: import sqlite3
import csv
import math
from pathlib import Path
import pandas as pd
from IPython.display import display, HTML

pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 160)

base_dir = Path.cwd()
movies_candidates = sorted(base_dir.rglob('movies.csv'))
ratings_candidates = sorted(base_dir.rglob('ratings.csv'))

if not movies_candidates or not ratings_candidates:
    raise FileNotFoundError('movies.csv or ratings.csv not found. Please check y

movies_path = movies_candidates[0]
ratings_path = ratings_candidates[0]

if movies_path.parent != ratings_path.parent:
    raise RuntimeError(f'movies.csv and ratings.csv are not in the same folder:

data_dir = movies_path.parent
db_path = base_dir / 'movielens.db'
results_dir = base_dir / 'query_results'
results_dir.mkdir(parents=True, exist_ok=True)

print('base_dir =', base_dir)
print('data_dir =', data_dir)
print('movies_path exists =', movies_path.exists())
print('ratings_path exists =', ratings_path.exists())
print('db_path =', db_path)
print('results_dir =', results_dir)
```

```
base_dir = c:\Users\19412\Desktop\3-2\database\proj3
data_dir = c:\Users\19412\Desktop\3-2\database\proj3\movielen数据集
movies_path exists = True
ratings_path exists = True
db_path = c:\Users\19412\Desktop\3-2\database\proj3\movielens.db
results_dir = c:\Users\19412\Desktop\3-2\database\proj3\query_results
```

```

In [2]: conn = sqlite3.connect(db_path)
conn.create_function('SQRT', 1, lambda x: None if x is None else math.sqrt(x))
cur = conn.cursor()

cur.executescript("""
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS ratings;
DROP TABLE IF EXISTS movie_genres;
DROP TABLE IF EXISTS movies;

CREATE TABLE movies (
    movieId INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    genres TEXT NOT NULL
);

CREATE TABLE ratings (
    userId INTEGER NOT NULL,
    movieId INTEGER NOT NULL,
    rating REAL NOT NULL,
    timestamp INTEGER,
    FOREIGN KEY (movieId) REFERENCES movies(movieId)
);

CREATE TABLE movie_genres (
    movieId INTEGER NOT NULL,
    genre TEXT NOT NULL,
    FOREIGN KEY (movieId) REFERENCES movies(movieId)
);
""")

movies_rows = []
movie_genre_rows = []
filtered_no_genre_tokens = 0
no_genre_tokens = {'no genre', 'no genres listed', '(no genres listed)'}

with movies_path.open('r', encoding='utf-8', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        movie_id = int(row['movieId'])
        title = row['title']
        genres = row['genres']
        movies_rows.append((movie_id, title, genres))
        for g in genres.split('|'):
            g = g.strip()
            g_norm = g.lower()
            if not g or g_norm in no_genre_tokens:
                filtered_no_genre_tokens += 1
                continue
            movie_genre_rows.append((movie_id, g))

cur.executemany('INSERT INTO movies (movieId, title, genres) VALUES (?, ?, ?)',
cur.executemany('INSERT INTO movie_genres (movieId, genre) VALUES (?, ?)', movie

ratings_rows = []
with ratings_path.open('r', encoding='utf-8', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:

```

```

        ratings_rows.append((
            int(row['userId']),
            int(row['movieId']),
            float(row['rating']),
            int(row['timestamp'])
        ))

cur.executemany('INSERT INTO ratings (userId, movieId, rating, timestamp) VALUES

cur.executescript("""
CREATE INDEX idx_ratings_movieId ON ratings(movieId);
CREATE INDEX idx_ratings_userId ON ratings(userId);
CREATE INDEX idx_movie_genres_movieId ON movie_genres(movieId);
CREATE INDEX idx_movie_genres_genre ON movie_genres(genre);
""")

conn.commit()

movie_cnt = cur.execute('SELECT COUNT(*) FROM movies').fetchone()[0]
rating_cnt = cur.execute('SELECT COUNT(*) FROM ratings').fetchone()[0]
movie_genre_cnt = cur.execute('SELECT COUNT(*) FROM movie_genres').fetchone()[0]

print('movies =', movie_cnt)
print('ratings =', rating_cnt)
print('movie_genres =', movie_genre_cnt)
print('filtered no-genre tokens =', filtered_no_genre_tokens)

```

```

movies = 10329
ratings = 105339
movie_genres = 23107
filtered no-genre tokens = 7

```

```

In [3]: # Data import validation
movie_cnt = pd.read_sql_query('SELECT COUNT(*) AS c FROM movies', conn)['c'].iloc[0]
rating_cnt = pd.read_sql_query('SELECT COUNT(*) AS c FROM ratings', conn)['c'].iloc[0]
movie_genre_cnt = pd.read_sql_query('SELECT COUNT(*) AS c FROM movie_genres', conn)['c'].iloc[0]
empty_genre_cnt = pd.read_sql_query(
    "SELECT COUNT(*) AS c FROM movie_genres WHERE genre IS NULL OR TRIM(genre) = ''",
    conn
)['c'].iloc[0]
placeholder_genre_cnt = pd.read_sql_query(
    """
    SELECT COUNT(*) AS c
    FROM movie_genres
    WHERE LOWER(TRIM(genre)) IN ('no genre', 'no genres listed', '(no genres listed)')
    """,
    conn
)['c'].iloc[0]

assert movie_cnt == 10329, f'movies row count mismatch: {movie_cnt}'
assert rating_cnt == 105339, f'ratings row count mismatch: {rating_cnt}'
assert movie_genre_cnt > movie_cnt, 'movie_genres row count should be greater than movies'
assert empty_genre_cnt == 0, f'Found empty genres: {empty_genre_cnt}'
assert placeholder_genre_cnt == 0, f'Found placeholder genres that should be excluded'

print('Data import validation passed')

```

Data import validation passed

```

In [4]: def _safe_name(name: str) -> str:
        return ''.join(ch if ch.isalnum() or ch in ('-', '_') else '_' for ch in name)

```

```

def run_sql(
    sql: str,
    query_name: str,
    preview_rows: int = 20,
    show_all: bool | None = None,
    save_csv: bool = True
) -> pd.DataFrame:
    df = pd.read_sql_query(sql, conn)

    if save_csv:
        file_name = _safe_name(query_name) + '.csv'
        out_path = results_dir / file_name
        df.to_csv(out_path, index=False, encoding='utf-8-sig')
        print(f'CSV saved: {out_path}')

    if show_all is None:
        show_all = len(df) <= 2000

    if show_all:
        html = df.to_html(index=False)
        display(HTML(f"<div style='max-height:480px; overflow:auto; border:1px solid #ccc; padding:5px;'>{html}</div>"))
        print('Display mode: full table in scrollable view')
    else:
        display(df.head(preview_rows))
        print(f'Display mode: preview first {preview_rows} rows')

    print(f'rows = {len(df)}')
    return df

```

1) Top 10 movies by average rating (minimum 10 ratings)

```

In [5]: q1_sql = """
SELECT
    m.movieId,
    m.title,
    COUNT(*) AS rating_cnt,
    ROUND(AVG(r.rating), 4) AS avg_rating
FROM ratings r
JOIN movies m ON m.movieId = r.movieId
GROUP BY m.movieId, m.title
HAVING COUNT(*) >= 10
ORDER BY avg_rating DESC, rating_cnt DESC, m.title ASC
LIMIT 10;
"""

df_q1 = run_sql(q1_sql, query_name='q1_top10_movies_by_avg_rating', preview_rows=10)
assert len(df_q1) == 10, f'Q1 result should have 10 rows, got {len(df_q1)}'

```

CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q1_top10_movies_by_avg_rating.csv

movieId	title	rating_cnt	avg_rating
1178	Paths of Glory (1957)	19	4.5000
1927	All Quiet on the Western Front (1930)	13	4.5000
1730	Kundun (1997)	10	4.5000
7099	Nausicaä of the Valley of the Wind (Kaze no tani no Naushika) (1984)	22	4.4773
1248	Touch of Evil (1958)	21	4.4762
3429	Creature Comforts (1989)	13	4.4615
1172	Cinema Paradiso (Nuovo cinema Paradiso) (1989)	37	4.4595
318	Shawshank Redemption, The (1994)	308	4.4545
66934	Dr. Horrible's Sing-Along Blog (2008)	23	4.4348
1949	Man for All Seasons, A (1966)	11	4.4091

Display mode: full table in scrollable view
rows = 10

2) Top 10 movies by average rating within each genre (minimum 10 ratings)

```
In [6]: q2_sql = """
WITH movie_genre_stats AS (
    SELECT
        mg.genre,
        m.movieId,
        m.title,
        COUNT(*) AS rating_cnt,
        AVG(r.rating) AS avg_rating
    FROM ratings r
    JOIN movies m ON m.movieId = r.movieId
    JOIN movie_genres mg ON mg.movieId = m.movieId
    GROUP BY mg.genre, m.movieId, m.title
    HAVING COUNT(*) >= 10
),
ranked AS (
    SELECT
        genre,
        movieId,
        title,
        rating_cnt,
        ROUND(avg_rating, 4) AS avg_rating,
        ROW_NUMBER() OVER (
            PARTITION BY genre
            ORDER BY avg_rating DESC, rating_cnt DESC, title ASC
        ) AS genre_rank
    FROM movie_genre_stats
)
SELECT *
FROM ranked
WHERE genre_rank <= 10
ORDER BY genre ASC, genre_rank ASC;
```

```

df_q2 = run_sql(q2_sql, query_name='q2_top10_movies_per_genre', preview_rows=40,

assert (df_q2['genre_rank'] <= 10).all(), 'Q2 has genre_rank > 10'
for genre, sub in df_q2.groupby('genre'):
    ranks = sorted(sub['genre_rank'].tolist())
    assert ranks == list(range(1, len(ranks) + 1)), f'Q2 genre {genre} rank is n

```

CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q2_top10_movies_per_genre.csv

genre	movieid	title	rating_cnt	avg_rating	genre_rank
Action	1927	All Quiet on the Western Front (1930)	13	4.5000	1
Action	3000	Princess Mononoke (Mononoke-hime) (1997)	52	4.3846	2
Action	3265	Hard-Boiled (Lat sau san taam) (1992)	13	4.3077	3
Action	908	North by Northwest (1959)	73	4.2740	4
Action	1224	Henry V (1989)	22	4.2727	5
Action	5782	Professional, The (Le professionnel) (1981)	22	4.2727	6
Action	2571	Matrix, The (1999)	261	4.2644	7
Action	1209	Once Upon a Time in the West (C'era una volta il	23	4.2609	8

Display mode: full table in scrollable view
rows = 190

3) Top 5 genres per user by overall evaluation (rating-count aware weighted score)

Overall score formula used in SQL

```

overall_score = ((rating_cnt * avg_rating) + (3 * user_avg_rating)) /
(rating_cnt + 3)

```

Term definitions

- **avg_rating** : this user's average rating within the current genre
- **rating_cnt** : how many ratings this user has in the current genre
- **user_avg_rating** : this user's baseline average rating across all rated movies
- **3** : smoothing strength (prior weight)

Low-count genres are pulled toward the user baseline, while high-count genres depend more on genre-specific ratings.

```
In [7]: q3_sql = """
WITH user_base AS (
    SELECT
        userId,
        AVG(rating) AS user_avg_rating
    FROM ratings
    GROUP BY userId
),
user_genre_stats AS (
    SELECT
        r.userId,
        mg.genre,
        COUNT(*) AS rating_cnt,
        AVG(r.rating) AS avg_rating
    FROM ratings r
    JOIN movie_genres mg ON mg.movieId = r.movieId
    GROUP BY r.userId, mg.genre
    HAVING COUNT(*) >= 2
),
scored AS (
    SELECT
        ugs.userId,
        ugs.genre,
        ugs.rating_cnt,
        ROUND(ugs.avg_rating, 4) AS avg_rating,
        ROUND(
            ((ugs.rating_cnt * ugs.avg_rating) + (3 * ub.user_avg_rating))
            / (ugs.rating_cnt + 3),
            4
        ) AS overall_score
    FROM user_genre_stats ugs
    JOIN user_base ub ON ub.userId = ugs.userId
),
ranked AS (
    SELECT
        userId,
        genre,
        rating_cnt,
        avg_rating,
        overall_score,
        ROW_NUMBER() OVER (
            PARTITION BY userId
            ORDER BY overall_score DESC, rating_cnt DESC, avg_rating DESC, genre
        ) AS rank_in_user
    FROM scored
)
SELECT *
FROM ranked
WHERE rank_in_user <= 5
ORDER BY userId ASC, rank_in_user ASC;
"""

df_q3 = run_sql(q3_sql, query_name='q3_top5_genres_per_user_weighted_overall', p
assert (df_q3['rank_in_user'] <= 5).all(), 'Q3 has rank_in_user > 5'
```

CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q3_top5_genres_per_user_weighted_overall.csv

	userId	genre	rating_cnt	avg_rating	overall_score	rank_in_user
0	1	Crime	31	4.2097	4.1584	1
1	1	War	10	4.2000	4.0681	2
2	1	Thriller	43	3.8721	3.8562	3
3	1	Drama	45	3.8444	3.8309	4
4	1	Action	46	3.8261	3.8140	5
5	2	Drama	11	4.3636	4.2635	1
6	2	Animation	2	4.5000	4.1379	2
7	2	Children	3	4.3333	4.1149	3
8	2	Crime	3	4.3333	4.1149	4
9	2	Fantasy	4	4.2500	4.0985	5
10	3	Mystery	4	4.2500	4.0548	1
11	3	Crime	12	4.0000	3.9589	2
12	3	Drama	36	3.9722	3.9586	3
13	3	Western	3	4.0000	3.8973	4
14	3	Horror	2	4.0000	3.8767	5
15	4	War	16	4.5625	4.4992	1
16	4	Animation	4	4.7500	4.4977	2
17	4	Mystery	10	4.4000	4.3449	3
18	4	Drama	76	4.3421	4.3352	4
19	4	Children	6	4.3333	4.2760	5
20	5	IMAX	9	4.2778	4.0043	1
21	5	Animation	21	4.0952	3.9813	2
22	5	Musical	11	4.0909	3.8965	3
23	5	Children	21	3.9048	3.8146	4
24	5	Fantasy	16	3.8438	3.7396	5
25	6	Romance	15	4.3667	4.3315	1
26	6	Crime	11	4.3636	4.3191	2
27	6	Sci-Fi	12	4.3333	4.2978	3
28	6	Musical	2	4.5000	4.2934	4
29	6	Drama	31	4.2581	4.2490	5

Display mode: preview first 30 rows
rows = 3340

4) Top 5 genres per user by watch count

```
In [8]: q4_sql = """
WITH user_genre_watch AS (
    SELECT
        r.userId,
        mg.genre,
        COUNT(DISTINCT r.movieId) AS watch_cnt,
        AVG(r.rating) AS avg_rating
    FROM ratings r
    JOIN movie_genres mg ON mg.movieId = r.movieId
    GROUP BY r.userId, mg.genre
),
ranked AS (
    SELECT
        userId,
        genre,
        watch_cnt,
        ROUND(avg_rating, 4) AS avg_rating,
        ROW_NUMBER() OVER (
            PARTITION BY userId
            ORDER BY watch_cnt DESC, avg_rating DESC, genre ASC
        ) AS rank_in_user
    FROM user_genre_watch
)
SELECT *
FROM ranked
WHERE rank_in_user <= 5
ORDER BY userId ASC, rank_in_user ASC;
"""

df_q4 = run_sql(q4_sql, query_name='q4_top5_genres_per_user_by_watch_count', pre
assert (df_q4['rank_in_user'] <= 5).all(), 'Q4 has rank_in_user > 5'
```

CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q4_top5_genres_per_user_by_watch_count.csv

	userId	genre	watch_cnt	avg_rating	rank_in_user
0	1	Action	46	3.8261	1
1	1	Drama	45	3.8444	2
2	1	Thriller	43	3.8721	3
3	1	Crime	31	4.2097	4
4	1	Adventure	31	3.6935	5
5	2	Thriller	12	3.9167	1
6	2	Drama	11	4.3636	2
7	2	Comedy	11	3.5455	3
8	2	Adventure	10	4.0000	4
9	2	Action	9	3.8889	5
10	3	Drama	36	3.9722	1
11	3	Comedy	35	3.7429	2
12	3	Romance	22	3.6364	3
13	3	Thriller	21	3.7143	4
14	3	Action	13	3.2308	5
15	4	Drama	76	4.3421	1
16	4	Comedy	46	4.0217	2
17	4	Romance	37	4.1622	3
18	4	Crime	18	3.9444	4
19	4	Thriller	18	3.7222	5
20	5	Comedy	45	3.4444	1
21	5	Adventure	22	3.5455	2
22	5	Animation	21	4.0952	3
23	5	Children	21	3.9048	4
24	5	Romance	21	3.7381	5
25	6	Drama	31	4.2581	1
26	6	Comedy	28	4.2143	2
27	6	Romance	15	4.3667	3
28	6	Adventure	14	4.2143	4
29	6	Thriller	13	4.0769	5

Display mode: preview first 30 rows
rows = 3340

5) Similar-interest user pairs

- Shared watch count per genre ≥ 3
- Number of qualified shared genres ≥ 2
- Stddev of rating differences on shared movies ≤ 0.3

```
In [9]: q5_sql = """
WITH common_movies AS (
    SELECT
        r1.userId AS user_a,
        r2.userId AS user_b,
        r1.movieId AS movieId,
        ABS(r1.rating - r2.rating) AS diff
    FROM ratings r1
    JOIN ratings r2
        ON r1.movieId = r2.movieId
        AND r1.userId < r2.userId
),
pair_genre_overlap AS (
    SELECT
        cm.user_a,
        cm.user_b,
        mg.genre,
        COUNT(*) AS overlap_cnt
    FROM common_movies cm
    JOIN movie_genres mg ON mg.movieId = cm.movieId
    GROUP BY cm.user_a, cm.user_b, mg.genre
    HAVING COUNT(*) >= 3
),
pair_genre_diff AS (
    SELECT
        cm.user_a,
        cm.user_b,
        mg.genre,
        SQRT(MAX(AVG(cm.diff * cm.diff) - AVG(cm.diff) * AVG(cm.diff), 0)) AS ra
    FROM common_movies cm
    JOIN movie_genres mg ON mg.movieId = cm.movieId
    GROUP BY cm.user_a, cm.user_b, mg.genre
),
qualified_pair_genres AS (
    SELECT
        o.user_a,
        o.user_b,
        o.genre,
        o.overlap_cnt,
        d.rating_diff_stddev
    FROM pair_genre_overlap o
    JOIN pair_genre_diff d
        ON o.user_a = d.user_a
        AND o.user_b = d.user_b
        AND o.genre = d.genre
    WHERE d.rating_diff_stddev <= 0.3
),
pair_summary AS (
    SELECT
        user_a,
        user_b,
        COUNT(*) AS qualified_genre_count,
        SUM(overlap_cnt) AS total_overlap_in_qualified_genres,
        ROUND(AVG(rating_diff_stddev), 4) AS avg_rating_diff_stddev,
        MIN(overlap_cnt) AS min_overlap_cnt,
```

```

        MAX(rating_diff_stddev) AS max_rating_diff_stddev,
        GROUP_CONCAT(genre, ', ') AS genre_list
    FROM qualified_pair_genres
    GROUP BY user_a, user_b
    HAVING COUNT(*) >= 2
)
SELECT *
FROM pair_summary
ORDER BY qualified_genre_count DESC,
        total_overlap_in_qualified_genres DESC,
        avg_rating_diff_stddev ASC,
        user_a ASC,
        user_b ASC;
"""

df_q5 = run_sql(q5_sql, query_name='q5_similar_interest_user_pairs', preview_row

if len(df_q5) > 0:
    assert (df_q5['user_a'] < df_q5['user_b']).all(), 'Q5 has mirrored duplicate
    assert (df_q5['qualified_genre_count'] >= 2).all(), 'Q5 has qualified_genre_
    assert (df_q5['min_overlap_cnt'] >= 3).all(), 'Q5 has genres with overlap_cn
    assert (df_q5['max_rating_diff_stddev'] <= 0.3 + 1e-12).all(), 'Q5 has ratin

print('Q5 threshold validation passed')

```

CSV saved: c:\Users\19412\Desktop\3-2\database\proj3\query_results\q5_similar_interest_user_pairs.csv

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_ratio
0	71	419	11		50
1	87	513	10		66
2	30	419	10		64
3	423	554	10		50
4	153	296	10		49
5	306	531	10		46
6	255	384	9		60
7	57	163	9		57
8	490	569	9		56
9	234	554	9		52

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_ratio
10	106	306		9	51
11	166	306		9	49
12	18	282		9	43
13	196	335		9	40
14	119	667		9	37
15	389	604		9	36
16	361	423		9	34
17	57	419		9	33
18	106	389		9	32
19	1	83		8	73

	user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_ratio
20	405	419	8	62	
21	57	628	8	56	
22	484	667	8	54	
23	83	642	8	46	
24	57	401	8	45	
25	296	314	8	45	
26	45	586	8	45	
27	238	365	8	43	
28	306	599	8	42	

user_a	user_b	qualified_genre_count	total_overlap_in_qualified_genres	avg_rating
29	57	607	8	40

Display mode: preview first 30 rows

rows = 21982

Q5 threshold validation passed

```
In [10]: # Optional: close the connection (skip this if you still want to query)
         # conn.close()
```